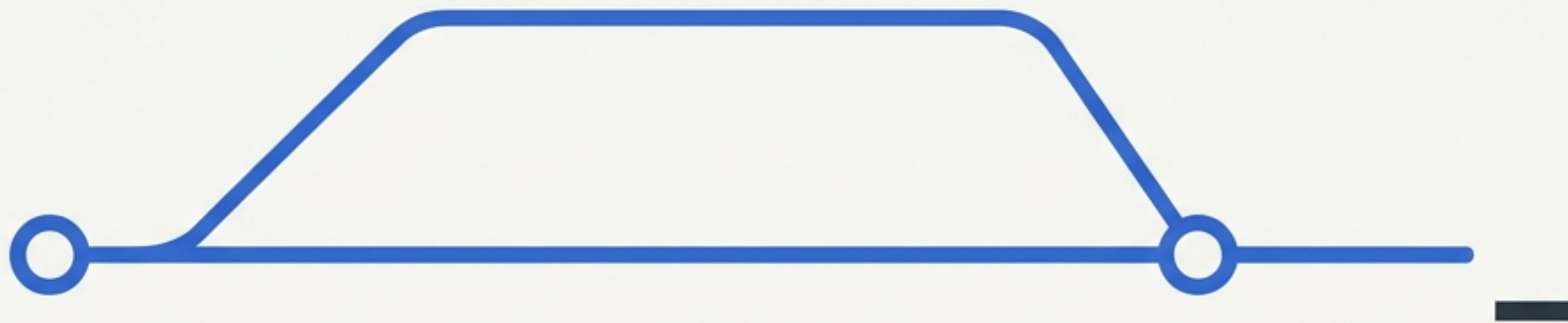
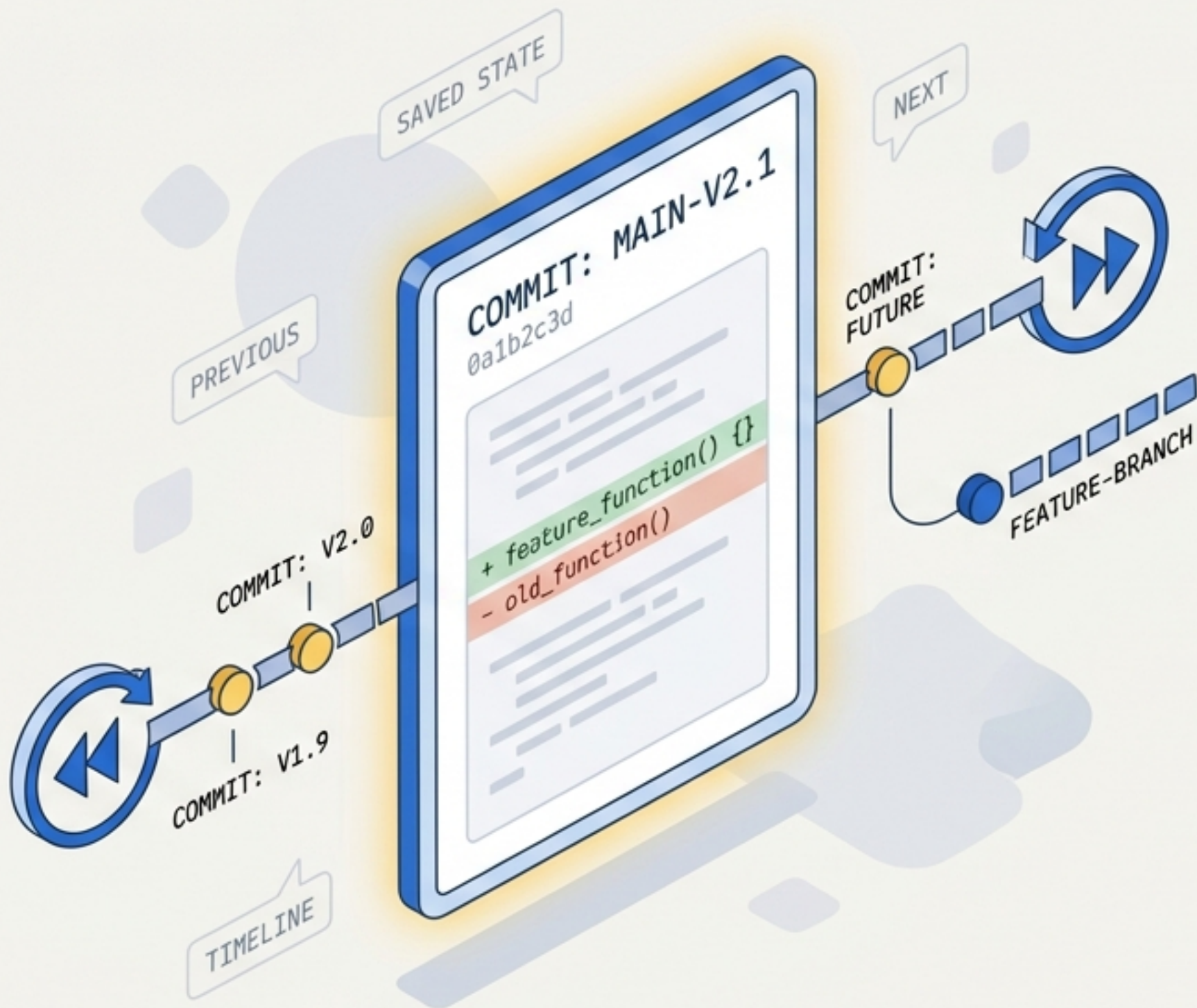




Git Workshop: Zero to Hero

(คู่มือฉบับเวิร์กชอป)

เข้าใจกระบวนการทำงานจริง พร้อมโค้ดตัวอย่างสำหรับผู้เริ่มต้น

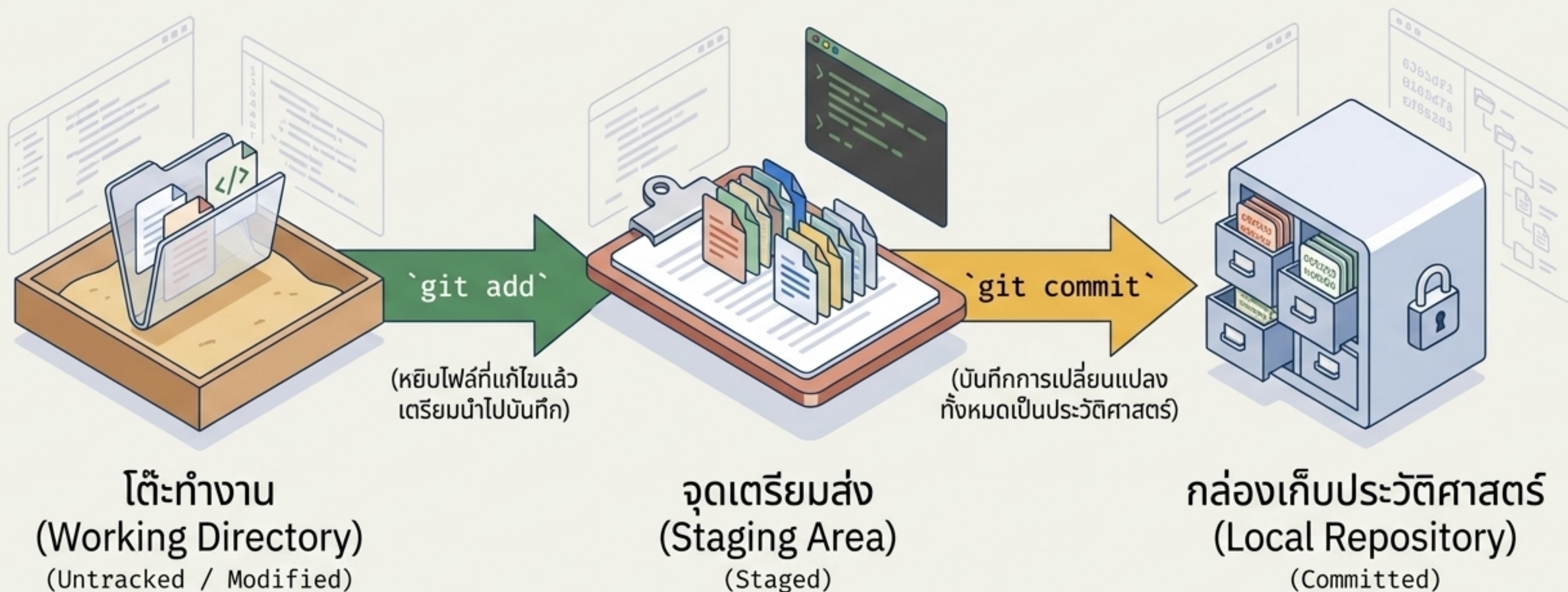


Git คือไทม์แมชชีน ของโปรแกรมเมอร์

ระบบควบคุมเวอร์ชันที่ช่วยให้คุณเซฟเกม
ได้ทุกจุด ทดลองสร้างฟีเจอร์ใหม่โดยไม่ทำ
ให้โค้ดหลักพัง และทำงานร่วมกับทีมพัฒนา
ได้ทั่วโลกอย่างปลอดภัย

การรักษาประวัติศาสตร์ของโปรเจกต์
คือหัวใจหลักของ Git

วงจรชีวิตของไฟล์ใน Git



Working Directory คือ Sandbox สำหรับทดลองแก้ไขโค้ด
ส่วน Staging Area คือด่านตรวจเช็ค (Checkpoint) ก่อนบันทึกจริง

1. เริ่มต้นสร้างโปรเจกต์ (Project Setup)

```
git init // สร้างคลังข้อมูลใหม่
```

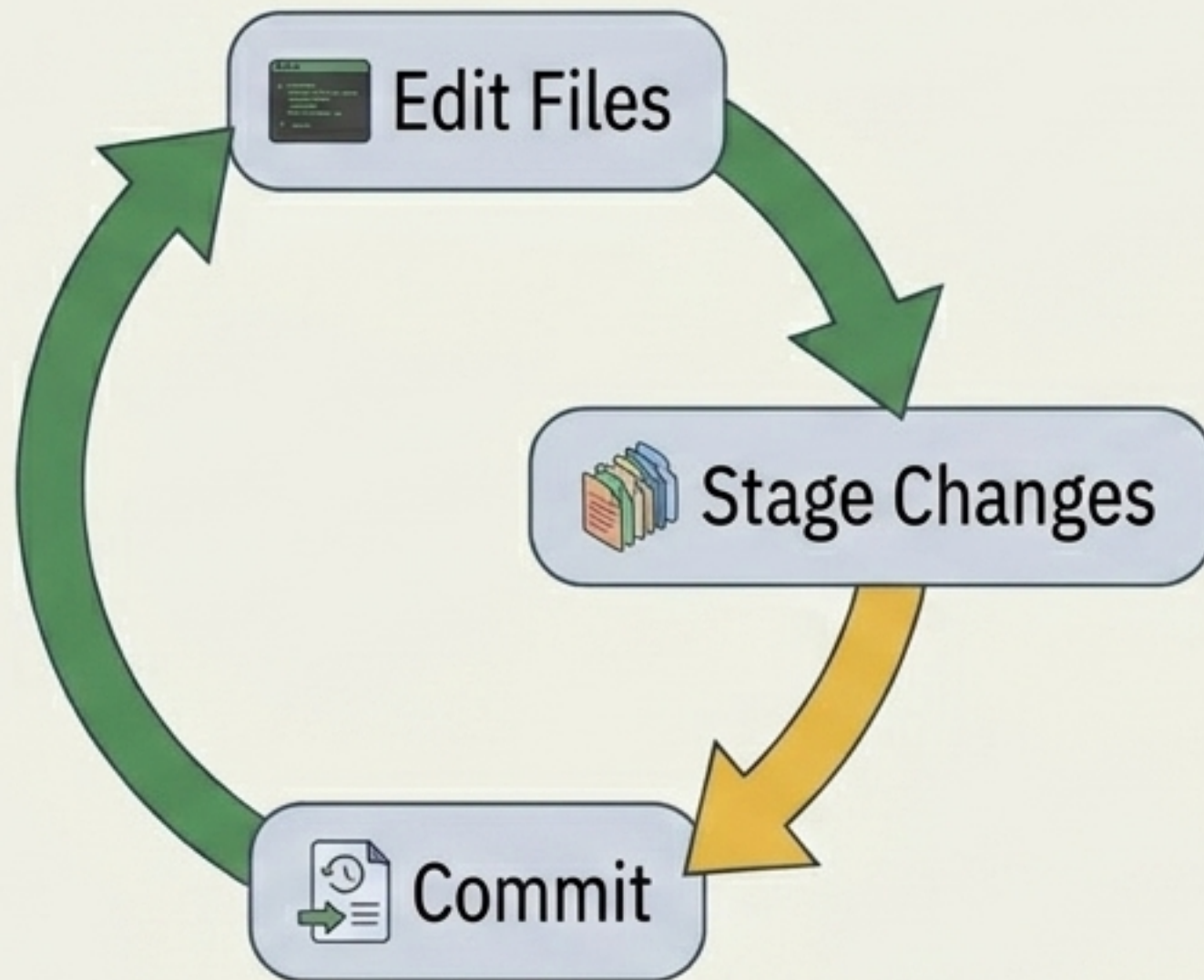
```
git config --global user.name "ชื่อของคุณ"
```

```
git config --global user.email "อีเมล@example.com"
```



ทำไมต้องตั้งค่า? Git ต้องรู้ว่าใครเป็นคนเขียนโค้ดชุดนี้ เพื่อให้ทีมสามารถตรวจสอบประวัติได้ถูกต้อง

2. วงจรการทำงานประจำวัน (The Daily Loop)



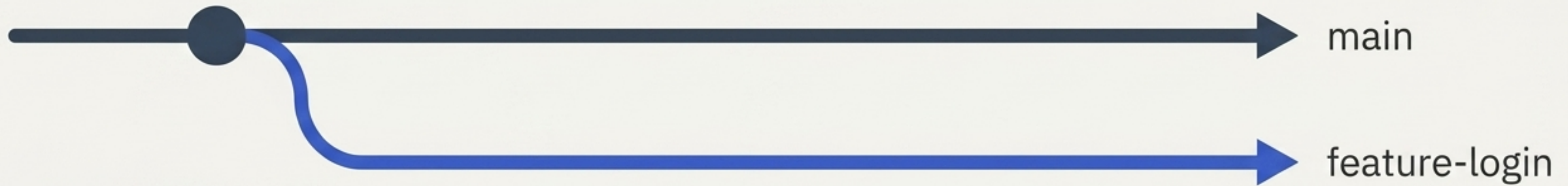
```
git status
// เช็คสถานะไฟล์เสมอ! ดูว่ามีไฟล์ไหนถูกแก้ไขบ้าง
git add index.html
// หยิบไฟล์เข้า Staging Area
git commit -m "Add homepage"
// บันทึกการเปลี่ยนแปลงพร้อมข้อความอธิบาย
```



Pro Tip: ข้อความ Commit ควรเขียนให้กระชับและอธิบายว่าเราเปลี่ยนแปลงอะไรลงไป

3. ทดลองโดยไม่กระทบของหลัก (Parallel Universes)

Branch คือการก๊อปปี้โค้ดไปสร้างจักรวาลคู่ขนานให้คุณทำงานอิสระโดยไม่กวนกัน

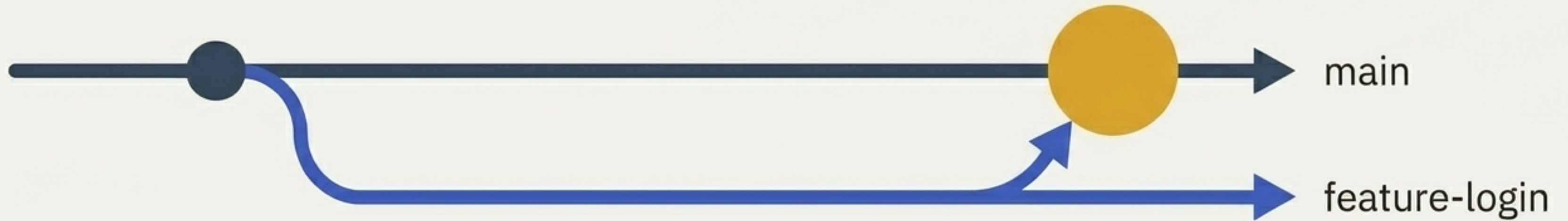


```
1 git branch feature-login // สร้างสาขาใหม่
2 git switch feature-login // สลับไปทำงานที่สาขานี้
```

📈 Level Up

ทางลัด! ใช้ ``git checkout -b <new_branch>`` เพื่อสร้างสาขาและสลับไปใช้งานทันทีในคำสั่งเดียว

4. รวมร่างโค้ด (Merging)

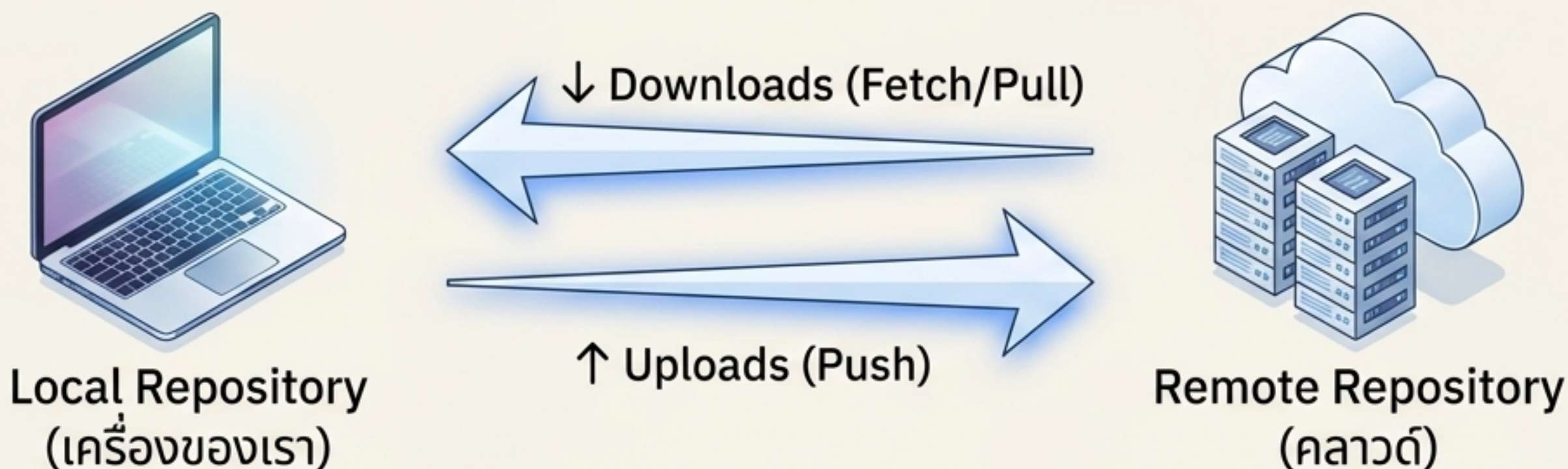


```
1 git switch main          // 1. ต้องกลับมาตั้งหลักที่สาขาหลักเสมอ
2 git merge feature-login  // 2. ดึงประวัติศาสตร์จากสาขาฟีเจอร์มารวม
```



ระวัง Merge Conflict! หากมีการแก้ไขไฟล์เดียวกัน บรรทัดเดียวกัน Git จะรวมให้อัตโนมัติไม่ได้
คุณจะต้องเป็นคนเลือกเองว่าต้องการเก็บโค้ดของใครไว้

5. ทำงานร่วมกับทีมบน Cloud (Remotes)



```
1 git clone <url> // คัดลอกโปรเจกต์จาก Cloud ลงเครื่อง
2 git fetch       // ตรวจสอบว่ามีอะไรอัปเดตใหม่ (แต่ยังไม่รวมเข้าโค้ดเรา)
3 git pull        // ดาวน์โหลดโค้ดใหม่และจับรวมเข้ากับเครื่องเราทันที
4 git push        // อัปโหลดโค้ดของเราขึ้นสู่ Cloud
```

``pull` = `fetch` + `merge``

6. ท่าไม้ตาย: การย้อนเวลา (Undo & Fix Mistakes)

ปัญหา (Problem)

พิมพ์ข้อความ Commit
ล่าสุดผิด!

เผลอแก้ไขไฟล์พัง
อยากให้กลับเป็นเหมือนเดิม!

อยากยกเลิก Commit นี้
แบบเนียนๆ โดยประวัติไม่พัง!

วิธีแก้ (Solution)

``git commit --amend``
(แก้ไข Commit ล่าสุด)

``git checkout -- <file>``
(ดึงไฟล์เก่าจาก Commit ล่าสุดมาทับ)

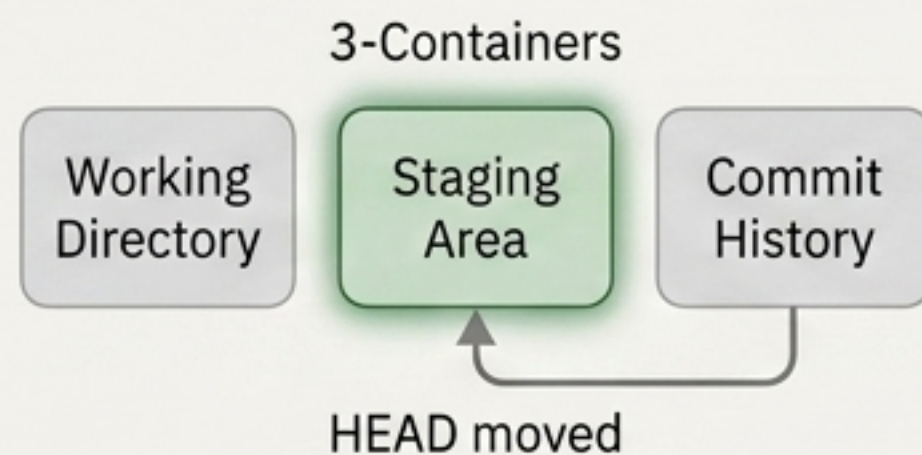
``git revert <commit>``
(สร้าง Commit ใหม่ที่มีผลตรงข้ามเพื่อหักล้างกัน)

เจาะลึกคำสั่ง git reset (การย้าย HEAD กลับไปอดีต)

`--soft` 🍃

ย้ายแค่ตำแหน่ง Commit.
โค้ดที่แก้แล้วยังอยู่ใน
Staging Area ครบถ้วน

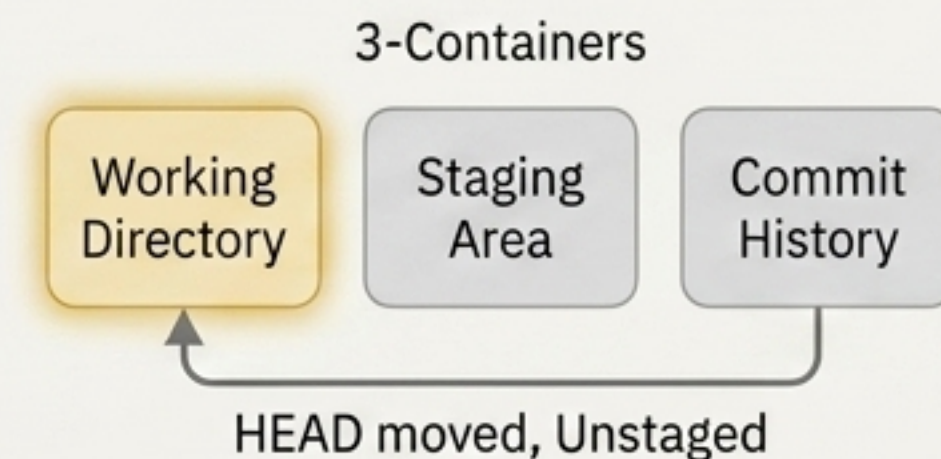
(เหมาะสำหรับจัดกลุ่ม
Commit ใหม่)



`--mixed` [Default] ✂️

ย้ายตำแหน่ง Commit และ
เอาของออกจาก Staging
Area แต่โค้ดยังอยู่บน
โต๊ะทำงาน

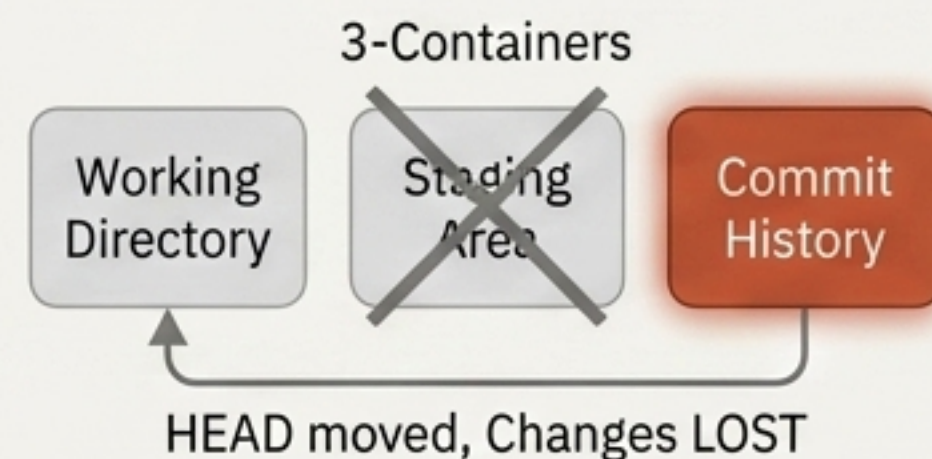
(เหมาะสำหรับแก้ไขโค้ดเพิ่มเติม
ก่อนจัดชุด Commit ใหม่)



`--hard` ⚠️

ย้ายตำแหน่ง Commit และ
ลบทุกอย่าง ที่เพิ่งแก้

(อันตรายมาก! โค้ดที่ยังไม่เซฟ
จะหายวับไปกับตา)



7. ช่อนงานชั่วคราว (Stashing)

กำลังเขียนโค้ดอยู่ แต่มีบั๊กฉุกเฉินแทรกเข้ามา?
ให้เก็บงานใส่ลิ้นชักไว้ก่อนโดยไม่ต้อง Commit ของที่ยังไม่เสร็จ!



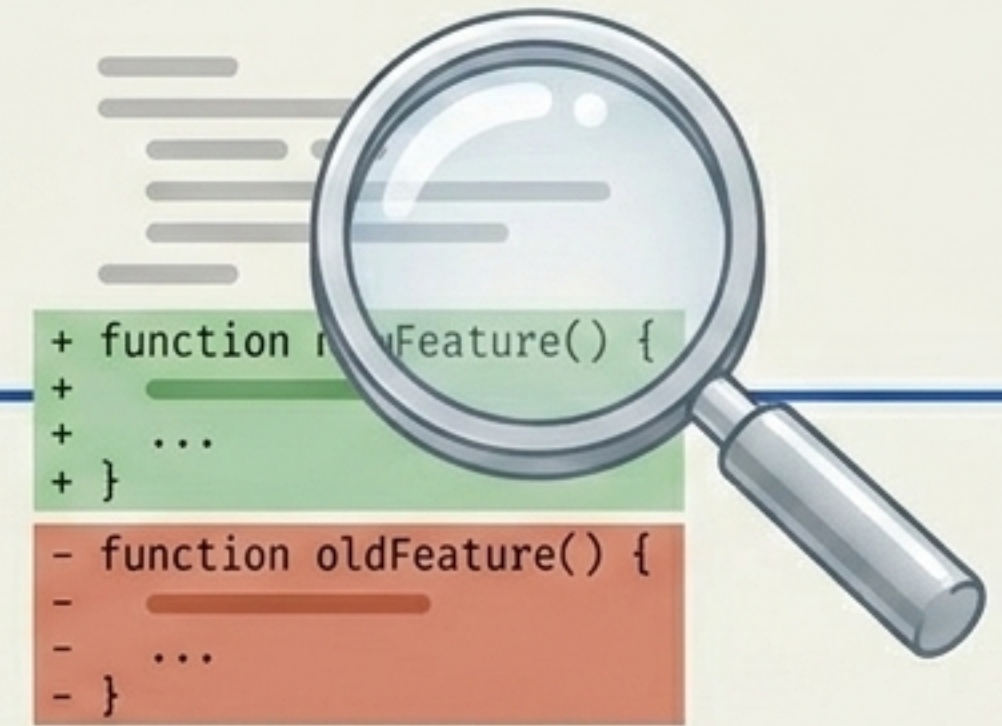
```
git stash  
// กวาดงานบนโต๊ะลงลิ้นชัก (ทำงานอื่นต่อได้เลย)
```

```
git stash list  
// ส่องดูว่ามีอะไรซ่อนอยู่ในลิ้นชักบ้าง
```

```
git stash pop  
// เอางานล่าสุดออกจากลิ้นชักมาวางบนโต๊ะทำต่อ
```


8. ส่องประวัติและการเปลี่ยนแปลง (Logs & Diffs)

```
$ git log --graph
.
| * eb3ca3b (HEAD -> main) K-noc
| \
| * 3a5c6d1 (HEAD -> main) Fix bug
| | |
| | /| 2b8e9f0 Add new feature
| /
| * d1c2e3b Initial commit
```



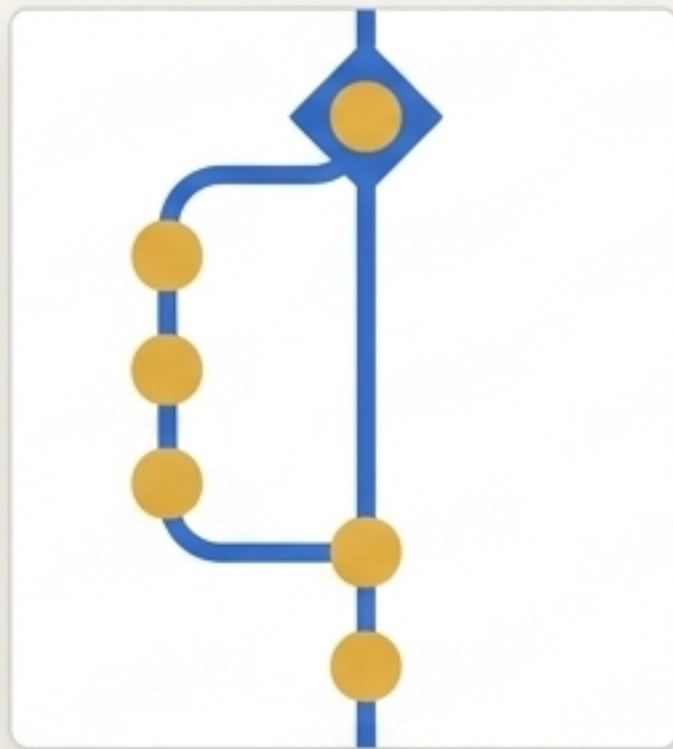
```
git diff
// ตรวจสอบแบบบรรทัดต่อบรรทัดว่าแก้ไขอะไรไปบ้างก่อนกด Add

git log --oneline --graph
// ดูประวัติ Commit ในรูปแบบสั้นๆที่อ่านง่ายสุดๆ

git blame <file>
// ตรวจสอบว่าใครเป็นคนเขียนโค้ดบรรทัดนี้... (หากันรับผิดชอบ!)
```

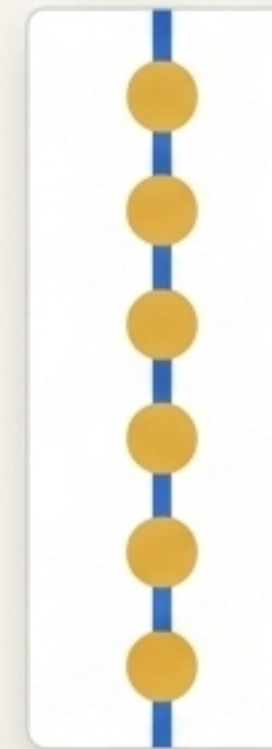

ขั้นกว่าของการรวมโค้ด: Merge vs Rebase

Merge (``git merge``)



สร้างจุดเชื่อมประวัติศาสตร์เข้าด้วยกัน ปลอดภัย
แต่เมื่อโปรเจกต์ใหญ่ขึ้น เส้นกราฟจะยุ่งเหยิง

Rebase (``git rebase``)



ยกชุด Commit ของเราไปต่อท้ายสุด
ประวัติสวยงามเป็นเส้นตรง อ่านง่ายสบายตา

กฎเหล็กของ Rebase: ห้ามใช้คำสั่งนี้กับ Branch สาธารณะที่มีคนอื่นกำลังใช้งานร่วมกับคุณเด็ดขาด!
(เพราะมันคือการเขียนประวัติศาสตร์ใหม่)

9. พิมพ์ให้ไวขึ้นด้วย Aliases (ทางลัด)



ขี้เกียจพิมพ์คำสั่งยาวๆ ซ้ำๆ ทุกวันใช่ไหม? คุณสามารถตั้งชื่อเล่นให้คำสั่ง Git ได้!

```
git config --global alias.st status // ต่อไปนี้พิมพ์แค่ `git st`  
git config --global alias.co checkout // ต่อไปนี้พิมพ์แค่ `git co`  
git config --global alias.ci commit // ต่อไปนี้พิมพ์แค่ `git ci`  
git config --global alias.br branch // ต่อไปนี้พิมพ์แค่ `git br`
```

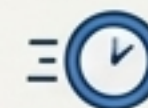
ก่อนใช้ Aliases

git checkout feature-branch

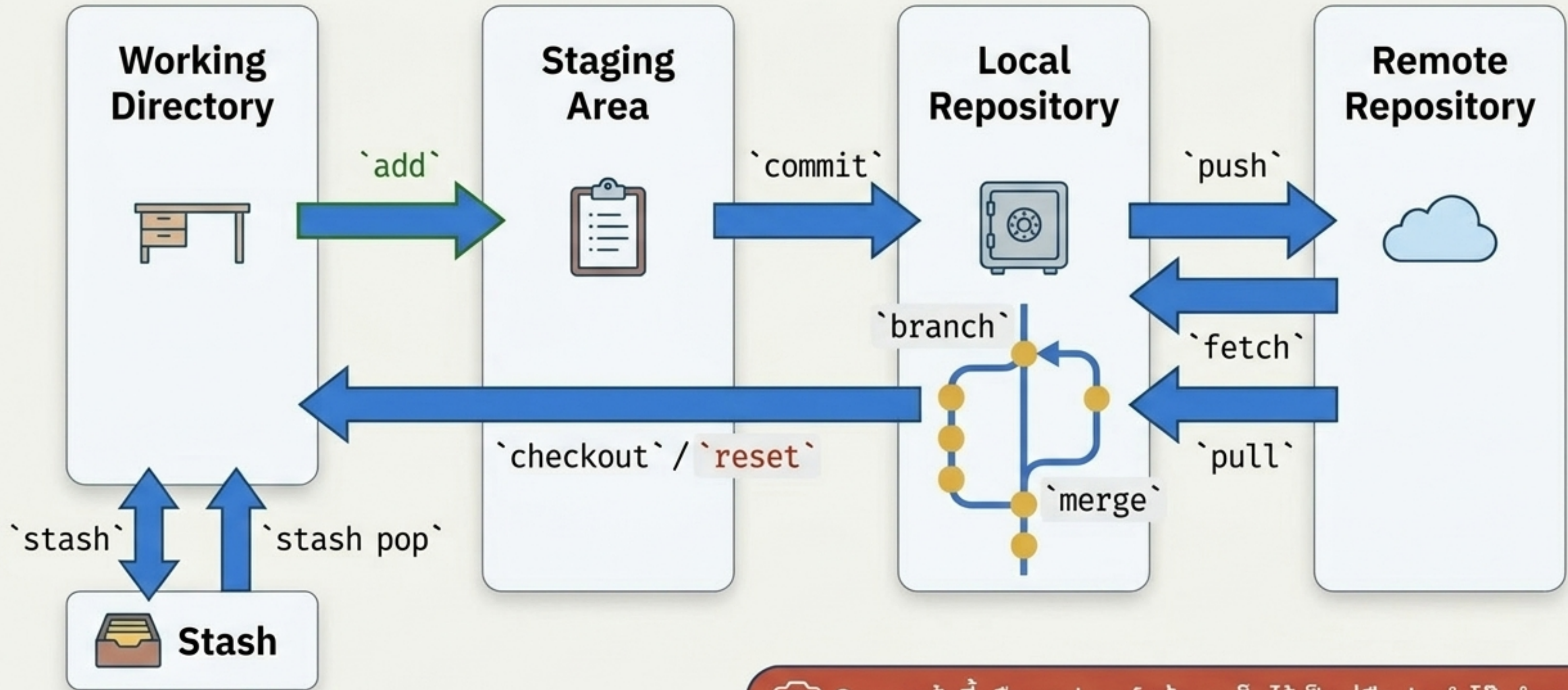


หลังใช้ Aliases

git co feature-branch



สรุปภาพรวม Workflow สำหรับมือโปร



📷 Save หน้านี้หรือแคปเจอร์หน้าจอเก็บไว้เป็นคู่มือประจำโต๊ะทำงานได้เลย!